

PROJECT_CONTEXT.md

AI Study Intelligence Platform

Production AI Document Intelligence + Adaptive Learning System

Version: 1.0

1. Project Vision

Build a production-ready AI-powered learning platform where users can upload large educational documents (PDFs), understand them faster using AI, chat with their documents, generate exams, track performance, and discover weak areas automatically.

This is not a simple PDF chatbot.

The system combines:

- Document Intelligence
- Retrieval Augmented Generation (RAG)
- Conversational AI Memory
- Automated Assessment Generation
- Learning Analytics
- Weak Topic Detection

The goal is to create an intelligent study companion similar in concept to modern AI document assistants, but focused on personalized learning and exam preparation.

2. Target Users

Primary:

Students Researchers Competitive exam candidates Professionals studying technical documents

User problems:

- Large PDFs take too long to study
 - Difficult to find important concepts
 - Hard to test understanding
 - No automatic feedback about weak areas
-

3. Core Features

Feature 1: Authentication System

Users can:

- Register
- Login
- Reset password
- Manage personal documents

Implementation:

Backend: FastAPI JWT authentication

Database: PostgreSQL

Tables:

users

Fields:

id name email password_hash created_at

Security:

Never store plain passwords.

Use bcrypt hashing.

Feature 2: PDF Upload System

User uploads PDF files.

Target:

Support:

- normal PDFs
- large PDFs
- academic documents

Flow:

Frontend

Upload PDF



FastAPI receives file



Save document



Create processing task



Background worker analyzes

Important:

Never block user while processing.

Return:

"Document processing started"

Processing status:

PENDING PROCESSING COMPLETED FAILED

Feature 3: Document Processing Pipeline

Pipeline:

PDF



Extract text



Clean text



Split into chunks

↓

Generate embeddings

↓

Store vectors

Libraries:

PDF extraction:

PyMuPDF

Backup:

pdfplumber

Chunking:

LangChain RecursiveCharacterTextSplitter

Recommended chunk:

size: 800-1000 tokens

overlap: 100-200 tokens

Feature 4: Vector Database System

Purpose:

Store document knowledge.

Options:

Development: ChromaDB

Production: Qdrant

Each vector stores:

embedding

metadata:

document_id user_id page_number chunk_text

This enables source-based answers.

Feature 5: AI Question Answering System

Use RAG architecture.

Flow:

User question

↓

Convert question into embedding

↓

Search similar document chunks

↓

Retrieve relevant context

↓

Send context + question to LLM

↓

Generate answer

Rules:

AI must answer from uploaded document.

If information does not exist:

Say:

"The document does not contain enough information."

Do not hallucinate.

Feature 6: Conversation Memory

Two memory systems.

Short Term Memory

Purpose:

Current conversation understanding.

Example:

User: Explain CNN

AI: Explains CNN

User: Give example

System understands: example of CNN

Long Term Memory

Store previous chats.

Database:

chat_history

Fields:

id

user_id

document_id

question

answer

created_at

User can reopen old conversations.

Feature 7: Answer Summary Button

Every AI answer contains:

Buttons:

Summarize

Simplify

Generate MCQ

Summary:

Input:

Long AI answer

Output:

Short key points

Do not run document retrieval again.

Use existing answer.

Feature 8: Automatic MCQ Generation Engine

User options:

Generate:

20 Questions

40 Questions

60 Questions

Difficulty:

Easy

Medium

Hard

Flow:

Document chunks



Important concept extraction



LLM generation



JSON validation



Save questions

MCQ format:

{ question:

option_a:

option_b:

option_c:

option_d:

correct_answer:

explanation:

topic: }

Feature 9: Exam Engine

User starts test.

Features:

Timer

Question navigation

Submit test

Calculate score

After submission:

Show:

Total questions

Correct answers

Wrong answers

Accuracy percentage

Topic performance

Feature 10: Weak Topic Detection

Purpose:

Personalized learning analytics.

Process:

Analyze wrong MCQ answers.

Group mistakes by topic.

Example:

Performance:

Machine Learning: 85%

Neural Networks: 40%

Statistics: 55%

Output:

Weak Topics:

1. Neural Networks
2. Statistics

Recommendation:

"Revise these topics again."

Backend Architecture

Technology:

Python

FastAPI

Folder:

backend/

app/

api/

auth/

database/

models/

schemas/

services/

rag/

mcq/

utils/

Rules:

No business logic inside routes.

Routes call services.

Services contain logic.

Database

Use:

PostgreSQL

Tables:

users

documents

chat_history

mcq_questions

exam_attempts

topic_scores

Use:

SQLAlchemy ORM

Alembic migrations

AI Stack

Framework:

LangChain

LLM support:

Gemini API

OpenAI API

Embedding:

sentence-transformers

or

OpenAI embeddings

Vector DB:

ChromaDB/Qdrant

Background Processing

Use:

Celery

Redis

Tasks:

PDF processing

Embedding generation

Summary creation

MCQ generation

Never process heavy jobs directly inside API request.

Frontend

Technology:

React

Pages:

Login

Register

Dashboard

Upload Document

Chat Interface

MCQ Test

Results

Weak Topics

UI Requirements:

Clean

Fast

Responsive

Professional SaaS style

API Design

Examples:

POST

/api/auth/register

POST

/api/auth/login

POST

/api/documents/upload

GET

/api/documents

POST

/api/chat/question

POST

/api/summary/generate

POST

/api/mcq/create

POST

/api/exam/submit

GET

/api/analytics/weak-topics

Performance Requirements

Large PDFs should not freeze UI.

Use:

Background workers

Pagination

Caching

Optimize:

Embedding creation

Database queries

Vector search

Error Handling

Handle:

Wrong file

Large file failure

AI API failure

Database failure

Return proper JSON errors.

Security

Use:

JWT tokens

Password hashing

Environment variables

Never expose:

API keys

Database passwords

Deployment Target

Docker based deployment.

Services:

Frontend container

Backend container

PostgreSQL

Redis

Vector Database

Optional:

AWS EC2

AWS S3

AWS RDS

Development Order

Phase 1:

FastAPI setup Database Authentication

Phase 2:

PDF upload Text extraction

Phase 3:

Chunking Embedding Vector storage

Phase 4:

RAG chat

Phase 5:

Memory system

Phase 6:

Summary generation

Phase 7:

MCQ generator

Phase 8:

Exam engine

Phase 9:

Weak topic analytics

Phase 10:

Docker deployment

AI Coding Agent Rules

When helping build this project:

Follow existing architecture.

Never randomly rewrite.

Never remove working features.

Generate production-level code.

Use environment variables.

Add comments only where needed.

Handle errors.

Think like a senior backend AI engineer.

Prioritize:

Correctness Security Scalability Maintainability

Resume Description After Completion

AI-Powered Adaptive Learning Platform

Built an AI document intelligence system using FastAPI, LangChain and RAG enabling users to analyze PDFs, query documents conversationally, generate assessments and identify weak learning areas.

Technologies:

Python FastAPI PostgreSQL LangChain Vector Database LLMs Docker AWS